

# Проект 4. Библиотеки NumPy и SciPy

---

В этом проекте ты познакомишься с библиотеками NumPy и SciPy — основными инструментами для работы с данными и статистикой в Python, а также проведёшь настоящий A/B-тест.

## Содержание

---

- [Глава 1](#)
  - [Про библиотеки NumPy и SciPy](#)
  - [Основные функции NumPy](#)
  - [A/B-тест: весь цикл от начала до конца](#)
- [Глава 2](#)
  - [Практические задания](#)
    - [Задание 1. Исследование данных и описательные статистики](#)
    - [Задание 2. Проведение A/B-теста \(t-тест\)](#)
    - [Задание 3. Анализ конверсии](#)

## Глава 1

### Про библиотеки NumPy и SciPy

Для начала работы в папке src скачай ноутбук project\_8.ipynb — это твоя рабочая тетрадь на проект, для удобства в ней можно работать и запускать код.

Ты уже знаком с несколькими библиотеками в Python, сегодня же мы продолжаем знакомство с другими библиотеками. В этом проекте ты освоишь две библиотеки — NumPy и SciPy.

#### NumPy

NumPy (Numerical Python) — это библиотека Python для работы с числовыми данными. Она позволяет быстро выполнять математические операции над большими массивами чисел.

#### Когда использовать NumPy

- Хранить и обрабатывать числовые данные (время, доход, оценки, метрики).
- Считать статистику: среднее, медиану, отклонение, перцентили.
- Выполнять математические преобразования: нормализация, логарифмирование.
- Подготовить данные для визуализации или машинного обучения.

#### SciPy

SciPy (Scientific Python) — это библиотека для научных и инженерных расчётов, построенная поверх NumPy. Она добавляет готовые инструменты для статистики, оптимизации, интеграции, обработки сигналов и многого другого.

#### Когда использовать SciPy

- Провести статистический тест (t-тест, хи-квадрат, корреляция).

- Сгенерировать данные из известного распределения (нормальное, биномиальное).
- Проверить гипотезу о данных (нормальность, равенство дисперсий).
- Решить задачу оптимизации или численного расчёта.

Представь, что у тебя задача — посчитать среднее время нахождения в приложении для 10 000 пользователей. Если бы не было библиотеки NumPy, пришлось бы писать цикл `for`, суммировать вручную данные и делить на количество. Работа шла бы медленно, и в таком коде легко было бы ошибиться. NumPy позволяет написать одну строку, и работа с данными будет происходить мгновенно. По такому же принципу библиотека SciPy ускоряет работу с данными.

Основные функции NumPy

| Функция                                       | Что делает                                                                                 | Зачем в А/В-тесте                                                                         | Пример кода                                              |
|-----------------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <code>np.array()</code>                       | Превращает в «быстрый массив», работает во много раз быстрее                               | Подготовить данные, чтобы потом быстро считать статистику                                 | <code>arr = np.array([25, 32, 18, 45])</code>            |
| <code>np.mean()</code>                        | Считает среднее арифметическое                                                             | Сравнить: в какой группе пользователи проводят больше времени / тратят больше денег       | <code>np.mean(group_a)</code>                            |
| <code>np.median()</code>                      | Находит «серединного» пользователя. Упорядочиваем все значения и берём то, что посередине. | Устойчива к выбросам: если один пользователь потратил 10 000 ₽, медиана не «поедет»       | <code>np.median(group_a)</code>                          |
| <code>np.std()</code>                         | Оценивает «разброс данных». Показывает, насколько данные разбросаны вокруг среднего.       | Понять, однородна ли аудитория: если показатель большой, значит пользователи очень разные | <code>np.std(group_a)</code>                             |
| <code>np.percentile()</code>                  | Отвечает на вопрос: «Ниже какого значения лежит X% данных?»                                | Узнать «типичный диапазон»: например, 25% пользователей заказывают от 25 до 45 мин.       | <code>np.percentile(data, 25)</code>                     |
| <code>np.min()</code> / <code>np.max()</code> | Показывает самое маленькое и самое большое значение в данных                               | Найти аномалии: отрицательное время? заказ на 100 000 ₽? — проверить данные               | <code>np.min(data)</code> ,<br><code>np.max(data)</code> |

Ниже ты увидишь, как сделать t-тест с бизнес-выводом:

```
# Импорт библиотеки
import numpy as np
```

```
# Данные: время доставки (в минутах) для 10 заказов
delivery_time_list = [25, 32, 18, 45, 28, 35, 22, 40, 30, 27]

# Преобразуем список в массив NumPy
delivery_time = np.array(delivery_time_list)

print("Тип данных:", type(delivery_time))
print("Массив:", delivery_time)
print("Форма массива:", delivery_time.shape)

# Считаем среднее и медиану
mean_delivery = np.mean(delivery_time)
median_delivery = np.median(delivery_time)

print("\n" + "="*50)
print("📊 СРЕДНЕЕ ВРЕМЯ ДОСТАВКИ")
print("="*50)
print(f"Среднее: {mean_delivery:.2f} минут")
print(f"Медиана: {median_delivery:.2f} минут")
print("\nЧто это значит:")
print(f"• В среднем курьер привозит заказ за {mean_delivery:.1f} мин.")
print(f"• Это можно использовать как базовый показатель сервиса")
print(f"• При обещании клиенту: 'доставим за {round(mean_delivery) + 10} мин.' — будет запас")
```

## А/В-тест: весь цикл от начала до конца

А/В-тест — это способ сравнить две версии продукта, чтобы понять, работает ли изменение на самом деле или разница возникла случайно.

Представь, что ты изменил кнопку в приложении с зелёной на красную. На следующий день заказов стало больше. Это из-за кнопки или просто так совпало?

**Шаг 1: Формулируем гипотезу** Пример: «Если мы внедрим новый алгоритм подбора курьеров, то среднее время доставки уменьшится на 15%, потому что система будет учитывать пробки в реальном времени»

### Шаг 2: Делим пользователей случайным образом

- Группа А (контрольная): 50% пользователей → старый алгоритм.
- Группа В (тестовая): 50% пользователей → новый алгоритм.

**Шаг 3: Собираем данные** Для каждого заказа фиксируем:

- в какой группе был заказ (А или В);
- время доставки (в минутах);
- дату и время заказа (чтобы сравнивать одинаковые условия).

**Шаг 4: Сравниваем группы** Пример данных после теста:

Группа А (старый алгоритм):

- Всего заказов: 1000.
- Среднее время доставки: 35 мин.

Группа В (новый алгоритм):

- Всего заказов: 1000.
- Среднее время доставки: 30 мин.

Вопрос: –5 минут — это реально или случайно?

### Шаг 5: Проверяем статистическую значимость

1. NumPy считает описательные метрики:
  - среднее, медиана, разброс в каждой группе.
2. SciPy проводит статистический тест (t-тест):
  - возвращает p-value — вероятность, что такая разница возникла бы случайно.
3. Интерпретируем результат:
  - $p < 0.05$  → «Маловероятно, что это случайность» → разница значима.
  - $p \geq 0.05$  → «Могло быть и случайно» → нельзя утверждать.

```
# A/B-тест: сравниваем алгоритмы подбора курьеров

# Импорт библиотек
import numpy as np
from scipy.stats import ttest_ind

# ШАГ 1: Формулируем гипотезу

print("="*70)
print("🎯 ШАГ 1: Гипотеза")
print("="*70)
print("""
Гипотеза: «Если мы внедрим новый алгоритм подбора курьеров,
           то среднее время доставки уменьшится на 15%,
           потому что система будет учитывать пробки в реальном времени».
""")

# ШАГ 2: Подготовка данных (в реальности – загрузка из файла)
print("="*70)
print("📦 ШАГ 2: Подготовка данных")
print("="*70)

# Абстрактные данные: время доставки (мин.) для двух групп
# В реальности: df = pd.read_csv('delivery_ab_test.csv')
group_a = np.array([32, 35, 28, 40, 33, 38, 31, 36, 29, 34,
                    37, 30, 35, 32, 39, 28, 33, 36, 31, 34]) # Старый
алгоритм
group_b = np.array([25, 28, 22, 30, 26, 29, 24, 27, 23, 25,
                    28, 24, 26, 29, 27, 23, 25, 28, 24, 26]) # Новый
алгоритм

print(f"Группа А (старый алгоритм): {len(group_a)} заказов")
```

```
print(f"Группа В (новый алгоритм): {len(group_b)} заказов")

# Проверка баланса групп
if len(group_a) == len(group_b):
    print("Группы сбалансированы по размеру")
else:
    print("Группы разного размера – интерпретируйте осторожно")

# ШАГ 3: Описательные статистики (NumPy)
print("\n" + "="*70)
print("ШАГ 3: Описательные статистики (NumPy)")
print("="*70)

def print_stats(name, data):
    """Выводит статистику для группы"""
    print(f"\n{name}:")
    print(f" • Среднее время доставки: {np.mean(data):.1f} мин.")
    print(f" • Медиана: {np.median(data):.1f} мин.")
    print(f" • Стд. отклонение: {np.std(data):.1f} мин.")
    print(f" • Мин/Макс: {np.min(data):.0f} / {np.max(data):.0f} мин.")
    print(f" • 25-й перцентиль: {np.percentile(data, 25):.0f} мин.")
    print(f" • 75-й перцентиль: {np.percentile(data, 75):.0f} мин.")

print_stats("Группа А", group_a)
print_stats("Группа В", group_b)

# ШАГ 4: Статистический тест (SciPy)
print("\n" + "="*70)
print("ШАГ 4: Проверка значимости (SciPy)")
print("="*70)

# Проводим t-тест
t_stat, p_value = ttest_ind(group_a, group_b, equal_var=False)

print(f"\nРезультаты t-теста:")
print(f" • t-статистика: {t_stat:.4f}")
print(f" • p-value: {p_value:.6f}")
print(f" • Порог значимости (alpha): 0.05")

# ШАГ 5: Интерпретация и бизнес-рекомендация
print("\n" + "="*70)
print("💡 ШАГ 5: Выводы и рекомендации")
print("="*70)

# Рассчитываем размер эффекта
mean_a = np.mean(group_a)
mean_b = np.mean(group_b)
abs_diff = mean_b - mean_a
rel_diff = (abs_diff / mean_a) * 100

print(f"\n📊 Размер эффекта:")
print(f" • Абсолютная разница: {abs_diff:+.1f} мин.")
print(f" • Относительная разница: {rel_diff:+.1f}%")
```

```
# Интерпретация p-value
print(f"\n🔍 Статистическая значимость:")
if p_value < 0.05:
    print(f"  p-value ({p_value:.4f}) < 0.05 → разница СТАТИСТИЧЕСКИ ЗНАЧИМА")
    print("  → Маловероятно, что такая разница возникла случайно")

# Направление эффекта
if mean_b < mean_a:
    print(f"\n🆕 НОВЫЙ АЛГОРИТМ РАБОТАЕТ ЛУЧШЕ!")
    print(f"  • Ускорение доставки: {abs(rel_diff):.1f}%")
    print(f"  • Экономия времени: {abs(abs_diff):.1f} мин. на заказ")
    print("\n👉 РЕКОМЕНДАЦИЯ: ВНЕДРИТЬ НОВЫЙ АЛГОРИТМ ДЛЯ ВСЕХ ПОЛЬЗОВАТЕЛЕЙ")
else:
    print(f"\n🆕 НОВЫЙ АЛГОРИТМ РАБОТАЕТ ХУЖЕ!")
    print(f"  • Замедление доставки: {abs(rel_diff):.1f}%")
    print("\n👉 РЕКОМЕНДАЦИЯ: ОТКАТИТЬ ИЗМЕНЕНИЕ ИЛИ ДОРАБОТАТЬ АЛГОРИТМ")
else:
    print(f"  p-value ({p_value:.4f}) ≥ 0.05 → разница НЕ ЗНАЧИМА")
    print("  → Такая разница могла возникнуть случайно")
    print("\n👉 РЕКОМЕНДАЦИЯ: СОБРАТЬ БОЛЬШЕ ДАННЫХ ИЛИ ПЕРЕСМОТРЕТЬ ГИПОТЕЗУ")
print("\n📊")
```

## Глава 2

### Практические задания

Данные датасета, которые необходимы в заданиях

| Поле             | Описание                                                         | Для чего используется                                  |
|------------------|------------------------------------------------------------------|--------------------------------------------------------|
| experimент_group | Группа эксперимента: А или В                                     | Деление пользователей на контрольную и тестовую группы |
| review_strategy  | Название стратегии повторения                                    | Описание различия между группами                       |
| p_recall_ab      | Доля правильного воспоминания слова после применения стратегии   | Основная числовая метрика для t-теста                  |
| retention_7d     | Вернулся ли пользователь к обучению на 7-й день: 1 — да, 0 — нет | Метрика конверсии для хи-квадрат теста                 |

### Задание 1. Исследование данных и описательные статистики

- Скачай файл duolingo.csv и загрузи данные.
- Выполни первичный осмотр данных:
  - Выведи размер датасета: количество строк и колонок.
  - Покажи первые 5 строк: df.head().

- Выведи названия колонок и типы данных.
  - Проверь пропущенные значения.
3. Выбери целевую метрику для анализа: `p_recall_ab`.
  4. Раздели данные на группы A и B по колонке.
  5. Проверь баланс групп: укажи количество записей в каждой группе.
  6. Посчитай описательную статистику через NumPy для каждой группы:
    - среднее;
    - медиану;
    - стандартное отклонение;
    - минимум и максимум;
    - 25-й и 75-й перцентили.
  7. Рассчитай разницу между группами:
    - абсолютная разница средних;
    - относительная разница в процентах.
  8. Сделай первичный вывод:
    - укажи, какая группа показывает большее значение;
    - попробуй объяснить, что это может значить для продукта.
  9. Сохрани файл `project_8_task_1.ipynb` и загрузи его в папку `src`.

## Задание 2. Проведение A/B-теста (t-тест)

1. Проведи t-тест для метрики `p_recall_ab`.
2. Интерпретируй p-value:
  - сравни p-value с порогом значимости 0.05;
  - сделай вывод: разница значима или может быть случайной.
3. Оцени размер эффекта:
  - рассчитай абсолютную разницу средних;
  - рассчитай относительную разницу в процентах;
  - оцени величину эффекта: малая <5%, умеренная 5–15%, большая >15%.
4. Сформулируй итоговый вывод:
  - Укажи, какая группа показывает лучший результат.
  - Укажи, можно ли доверять этому результату.
5. Опционально визуализируй распределение:
  - построй гистограммы для групп A и B;
  - добавь вертикальные линии средних значений.
6. Сохрани файл `project_8_task_2.ipynb` и загрузи его в папку `src`.

## Задание 3. Анализ конверсии

1. Посчитай конверсию в каждой группе, выведи результат в процентах для группы A и группы B.

Тут конверсия — это доля пользователей или наблюдений, где `retention_7d = 1`.

2. Проведи тест для сравнения долей: хи-квадрат тест.
  - Создай таблицу сопряжённости.
  - Выведи p-value в консоль.
3. Интерпретируй результат:

- сравни p-value с порогом 0.05;
- сделай вывод: разница в конверсии значима или может быть случайной.

4. Сформулируй итоговую рекомендацию:

- если разница значима и конверсия  $B > A \rightarrow$  «Рекомендуем внедрить версию B»;
- если разница значима и конверсия  $B < A \rightarrow$  «Рекомендуем доработать версию B»;
- если разница не значима  $\rightarrow$  «Недостаточно данных для вывода, протестируем ещё раз».

5. Сохрани файл project\_8\_task\_3.ipynb и загрузи его в папку src.